

Parmana: Policy-Checked Execution with Signed Human Approval and Offline-Verifiable Records for AI Agent Actions

Pavan Dev Singh Charak, Parmana Systems

6 October 2026

Also on the docs site: <https://docs.parmanasystems.com/evaluation/paper>. Every number and claim is taken from [CLAIMS.md](#), [THREAT-MODEL.md](#), [VERIFICATION-GAPS.md](#) and [MUTATION-TESTING.md](#) as of the date above; those files are the current record.

Abstract

AI agents now call tools that move money, merge code and change customer records. The usual controls answer narrower questions: identity and access management decides who may call an API, and guardrails decide whether text looks harmful. Neither decides whether one specific action, with its exact parameters, may run now, and neither leaves evidence a third party can check without trusting the operator. We describe Parmana, a server that sits between an agent and the systems it acts on. Every proposed action is evaluated against a versioned, deterministic policy; an action whose policy requires it runs only with a signed, single use, time bounded approval from a registered person, bound to the action, the resource and the amount; release happens only at a gateway that re-verifies a signed authorization and holds the credentials the agent never receives; and every action and every refusal yields a signed record that verifies offline with only a public key. We state a threat model with 18 threats and seven explicit assumptions, back it with 16 runnable attack scenarios, mutation testing of the security-critical packages (83.6% to 96.0% mutation score) and property-based fuzzing, and publish the open gaps alongside the claims. We report what the design does not do: it does not detect prompt injection, it governs only actions routed through it, and an approval covers the action, resource and amount but not every parameter.

1. Introduction

An agent that can issue a refund, merge a pull request or post to a customer channel turns a model error, or a successful prompt injection, into an action with consequences. Public incidents show the pattern: an agent told not to change anything without approval deleted a production database [Replit 2025]; a prompt inserted into a coding extension instructed the assistant to delete files and cloud resources [Amazon Q 2025]. In both, what the agent could do was decided by the credentials it held, and the instruction "ask first" lived only in the conversation.

Three properties are missing from the usual stack:

1. **Per action authorization.** OAuth scopes and roles grant classes of actions for long periods [RFC 6749]. They do not express "this refund, to this customer, up to this amount, once, in the next fifteen minutes, approved by this person".
2. **Complete mediation of the action, not the text.** Guardrails classify model input and output. An agent that holds credentials can act whatever a classifier said [Greshake 2023].
3. **Evidence that does not depend on the operator.** Logs are trusted as stored. An auditor, regulator or customer should be able to check what ran, and what was refused, with no access to the operator's systems.

Parmana addresses these for actions routed through it. Its contributions, as implemented and deployed:

- A pipeline in which no agent action is authorized without a signed human approval bound to the action, the resource and, where the policy names one, the amount (Section 4.3).
- A gateway that is the sole release point, re-verifies a signed, single use, content bound authorization before every release, and holds all downstream credentials (Section 4.4).
- Signed records of every action and every refusal, with a signed intent written before release, verifiable offline by an open source verifier (Section 4.5).
- Governance of policies, approvers and connectors through maker checker with signed step up approvals (Section 4.6).
- An evaluation method that ties every claim to a test, a runnable attack scenario or a stated gap, including the gaps found by mutation testing and fuzzing (Section 6).

2. Background and related work

Reference monitors and separation of duty. Complete mediation and least privilege [Saltzer and Schroeder 1975] and the separation of duty in well formed transactions [Clark and Wilson 1987] are the oldest ideas used here. Parmana applies them to a new subject: an automated caller that is assumed hostile.

Authorization languages. Policy engines such as OPA and Cedar [Cutler et al. 2024] decide allow or deny from inputs. Parmana's policy language is deliberately smaller (first matching rule wins; no matching rule refuses) and adds two things engines leave to the caller: a required signed human approval as a condition, and a signed decision that the release point re-verifies.

Capability tokens. Macaroons [Birgisson et al. 2014] attach caveats to bearer tokens. Parmana's execution authorization is a signed, single use, time bounded token bound to the hash of the exact request, verified at the gateway and spent in a shared nonce store.

Tamper-evident logging. Secure audit logs [Schneier and Kelsey 1999], tamper-evident data structures [Crosby and Wallach 2009] and Certificate Transparency [RFC 6962] make logs verifiable. Parmana signs each record individually and chains executions and caller audit events.

Defences against prompt injection. Work on prompt injection [Greshake et al. 2023] and on designs that separate control from data flows [Debenedetti et al. 2025] aims to keep an injected instruction from steering the agent. Parmana is complementary: it assumes the agent may be steered and limits what a steered agent can make happen.

3. Threat model

The full model is published as THREAT-MODEL.md. In summary:

System. The API, the runtime that evaluates policy and signs authorizations, the execution gateway and execution control, built in connectors (HubSpot, GitHub, Slack, Paytm), release to registered external endpoints, governance, and the signed records. Out of scope: downstream systems, the agent and its model, the hosting platform, and the people who approve.

Trust boundaries. Seven boundaries (Figure 1): caller to API (B1), approver to server (B2), governance (B3), decision to release (B4), release to connector (B5), server to database (B6), and record to verifier (B7).

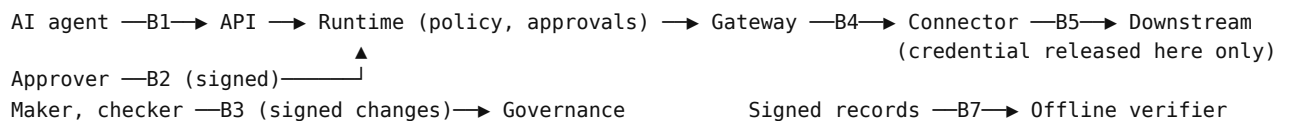


Figure 1. Trust boundaries.

Attackers in scope. A0, anyone on the network; A1, a caller with a valid agent key who controls the whole request, including a prompt injected or compromised agent; A2, an attacker holding a captured authorization or used approval; A3, a holder of one governance credential; A4, an attacker who can read or alter stored records but holds no signing key.

Assumptions, stated rather than defended. (1) Signing keys are not compromised; (2) approver and step up keys are held by the people they name; (3) maker and checker are two people; (4) the database is not writable by an attacker; (5) connector credentials are held only by execution control; (6) downstream systems accept actions only from Parmana or verify its signed release; (7) policies are written correctly. Parmana proves which policy decided, not that it is right.

Threats. Eighteen threats T1 to T18, each with control, evidence and residual risk. Examples: T2, a prompt injected agent requests an action no person approved; T4, declared facts describe one action while another executes; T6, a forged, expired or replayed authorization; T9, a code path reaches a connector without the gateway; T12, a signed record is altered.

4. Design

4.1 The request

A caller submits a business transaction: the intent (an action such as `paytm:refund`, a target, parameters), the facts it declares (signals), and optionally signed approvals. The caller's API key is limited to named capabilities, principals and tenant (E1).

4.2 Policy evaluation

Each capability is bound to exactly one policy. A policy is a versioned JSON file: typed signals (`signalsSchema`), approval signals, and ordered rules whose conditions compare signals. The first matching rule decides; no matching rule refuses. Evaluation is deterministic: no clock, randomness or I/O inside the engine. Before any rule runs, every declared signal must have its declared type (an amount sent as text is refused), and signals the policy binds to the intent must equal the intent's own values, so a caller cannot declare one action and execute another.

4.3 Signed human approval

A policy's approve rule must require an approval signal as its whole condition or directly in its top level conjunction; a policy that could approve without one fails validation at load and cannot be proposed. An approval signal counts as true only with a signed approval that is Ed25519 signed by a registered, unrevoked approver; bound to the action, the resource and the amount where the policy declares one; unexpired (15 minutes by default, at most 24 hours); and single use. It is checked at decision and again at release. Facts the agent declares can only cause a refusal, never an approval.

4.4 Authorization and release

An approved decision yields an execution authorization: signed, single use, time bounded, and bound to the canonical hash of the exact request (canonical JSON with sorted keys, SHA-256). The gateway is the only component that can call a connector. Before every release it verifies the signature, the expiry to the instant, the content hash, that the policy named is the approved current version, the signals, and the nonce, spent atomically in a shared Postgres store so each authorization is used once across instances. Execution control then issues a single use session and resolves the connector credential for that call only; credentials are never in the agent's hands, never stored as state, and are redacted to a fingerprint in records. An architecture test over package boundaries checks that no other path reaches a connector.

4.5 Records

Before release, a signed Execution Intent is stored; if it cannot be stored, nothing is released. After release, a signed Execution Trust Record holds the decision, the policy and its version, the approval, the authorization and the result; if it cannot be written, it is rebuilt from the context saved right after release. A refusal yields a signed Refusal Record. Executions are hash chained, and caller authentication events form a per caller chain. Every field is in the signed canonical bytes, so any change fails offline verification.

Signatures are Ed25519 [RFC 8032], optionally together with ML-DSA-65 [FIPS 204] as a hybrid pair with distinct algorithms. Keys are local files or AWS KMS. Because KMS refuses raw Ed25519 messages over 4096 bytes, longer messages are signed as a fixed commitment (a domain prefix and the SHA-512 digest); a verifier accepts the commitment form only above that length, so a small message cannot be downgraded.

Verification needs only the record and the public key: the open source `@parmana/sign` package, a Python module, or a page in the browser. A valid result proves the record is unchanged and was signed by the key holder; it does not prove the key holder was honest.

4.6 Governance

Policies, approvers and external connectors change only through maker checker: a proposal by one human credential and an approval by a second, carrying a step up signature bound to that change. The change and its approval are signed and chained. The gateway refuses an authorization whose policy is not the approved current version.

5. Implementation and deployment

Parmana is a TypeScript monorepo (Node 24) with TypeScript and Python SDKs and connector SDKs. It runs as a hosted API (production mode, AWS KMS or local keys, Postgres), self hosted with Docker Compose, and as a public sandbox that acts on nothing and uses a published demo key. Since 30 September 2026 every agent action in production requires a signed approval, reads included, after 13 policy versions were approved through maker checker.

6. Evaluation

6.1 Attack scenarios

Sixteen scenarios, each runnable with `npm run evaluate -- EV-xx`, attack the controls behind T1 to T13. Each passes only when the attack is refused or detected.

ID	Attack	ID	Attack
EV-01	Replay a used authorization	EV-09	Rely on facts that changed after approval
EV-02	Forge or alter an authorization	EV-10	Obtain a connector's credentials
EV-03	Expired authorization or revoked key	EV-11	Reach a connector without the gateway
EV-04	Act without a signed approval	EV-12	Act without credentials or outside scope
EV-05	Forge or misuse an approval	EV-13	Change a policy without a second person
EV-06	Declare one action, execute another	EV-14	Alter a signed record
EV-07	Be judged by a weaker policy	EV-15	Run one transaction twice by racing
EV-08	Execute under an outdated policy	EV-16	Policy file outside the policy directory

6.2 Mutation testing

Stryker mutation testing of the five security-critical packages, re-scored on Node 24 after writing tests for the surviving mutants:

Package	First run	After
envelope-verifier	89.7%	94.0%
approval	77.7%	93.9%
policy	50.3%	96.0%
execution-gateway	67.6%	83.6%
crypto	65.4%	85.0%

Surviving mutants led to real fixes, not only tests. One example (G-89): an edited execution with its chain signature removed passed the chain check, because a partially chained execution was treated as legacy data. It still failed the record's own signature, so it was not exploitable alone, but the second layer was weaker than described; it now fails.

6.3 Fuzzing

Property-based fuzz tests hand the API, the policy validator, the approval verifier and the offline verifier arbitrary inputs and require a clear refusal, never a crash. They found a request shape that crashed validation instead of returning a 400 (G-87), a malformed policy that threw instead of being refused (G-90), and a malformed signatures field that made the offline verifier throw (G-91). Each is fixed with a test that failed before.

6.4 Static analysis and disclosure

CodeQL on every change found a path traversal: a policy name or version of . . let the file based policy store read and write outside its directory (G-88). It required an authenticated proposer and a second approver; it was fixed, refused earlier at proposal time, and published as a low severity advisory (GHSA-hp43-9p54-wqp9).

6.5 Claims and gaps

Every claim in CLAIMS.md names its code and tests and states its scope. Gaps are recorded in VERIFICATION-GAPS.md with an "open issues at a glance" table, including the two that matter most here: approvers are not yet limited to particular actions or policies (G-50), and facts an agent declares are not checked against another system (G-51).

7. Limitations

- **Only actions routed through Parmana.** It enforces nothing at the network level. An agent that holds other credentials can act outside it.
- **No text inspection.** Parmana does not detect prompt injection, harmful output or leakage.
- **Approval granularity.** An approval fixes the action, the resource and the amount, not every parameter: a Slack approval does not fix the message text.
- **People.** A person can be persuaded to approve. In the current production deployment one person holds both maker and checker credentials and the only approver key; the server cannot tell two credentials belong to two people.
- **Key and database compromise** are assumed away; with both, history can be rewritten. Deleting a caller's whole audit history, or reordering rows across callers, is not caught by the per caller chain.
- **Operations.** No metrics or dashboards; key custody is local files or AWS KMS only.

8. Conclusion

Parmana moves the decision about an agent's action out of the conversation and into a checked, signed step that a person approves and a third party can verify. It does not make an agent safe; it bounds what an agent can make happen, for the actions routed through it, and leaves evidence that does not depend on trusting the operator. The claims, the attacks and the gaps are public so that each can be checked.

References

- Birgisson, A., Politz, J. G., Erlingsson, U., Taly, A., Vrabie, M., Lentczner, M. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. NDSS 2014.
- Clark, D. D., Wilson, D. R. A Comparison of Commercial and Military Computer Security Policies. IEEE Symposium on Security and Privacy, 1987.
- Crosby, S. A., Wallach, D. S. Efficient Data Structures for Tamper-Evident Logging. USENIX Security 2009.
- Cutler, J. W., et al. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. OOPSLA 2024.
- Debenedetti, E., et al. Defeating Prompt Injections by Design. arXiv, 2025.
- Greshake, K., et al. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. AISec 2023.
- Hardt, D. The OAuth 2.0 Authorization Framework. RFC 6749, 2012.
- Josefsson, S., Liusvaara, I. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, 2017.
- Laurie, B., Langley, A., Kasper, E. Certificate Transparency. RFC 6962, 2013.
- NIST. Module-Lattice-Based Digital Signature Standard. FIPS 204, 2024.
- Saltzer, J. H., Schroeder, M. D. The Protection of Information in Computer Systems. Proceedings of the IEEE 63(9), 1975.
- Schneier, B., Kelsey, J. Secure Audit Logs to Support Computer Forensics. ACM TISSEC 2(2), 1999.
- [Replit 2025] Fortune, 23 July 2025; AI Incident Database, incident 1152.
- [Amazon Q 2025] TechRepublic and Nudge Security reports on Amazon Q Developer for VS Code 1.84.0, July 2025.